
Not-Normal: *Not a Dashboard*

Technical Report for the UNOCHA Project

Juri Taresch Leonard Küch Oliver Zimmermann

Abstract

We present *Not a Dashboard*, an interactive platform for quickly exploring arbitrary CSV-based tabular data with lightweight machine learning. The system is designed for users with limited technical background: after uploading a dataset, they can inspect the data, define a target, choose explanatory variables, configure feature encodings, train a model, review diagnostics, and run single-row predictions with uncertainty intervals. A central contribution is the *Pitfalls* panel, which translates model state and quality signals into adaptive warnings that are easier for non-experts to act on. Our goal is not to automate decisions or claim perfect predictions, but to provide an extensible decision-support platform that helps humanitarian users explore data faster, more transparently, and with clearer awareness of uncertainty and misuse risk.

1 Introduction

Decision makers often have access to tabular data but not to the time, tooling, or technical expertise needed to turn that data into useful evidence quickly. Static dashboards are too rigid for heterogeneous CSV inputs, while programming-based analysis environments are too demanding for many operational users. In the UNOCHA setting, this creates a practical gap: users need a way to explore unfamiliar data, test hypotheses, and obtain cautious model-based signals without being forced into a full data science workflow.

We address this gap with *Not a Dashboard*, a generic upload-driven modeling workbench for tabular data. The platform supports CSV upload, lightweight inspection, user-defined target and feature selection, configurable encodings, model training, holdout evaluation, diagnostics, uncertainty-aware prediction, and adaptive warnings. It is explicitly designed for *decision support*, not automated decision-making.

2 Motivation and Initial Data Exploration

First, we started by diving deeper into the funding process of OCHA. During our research, we discovered the Underfunded Emergencies (UFE) initiative, which focuses on addressing humanitarian crises that receive insufficient funding and provides support via allocations from the Central Emergency Response Fund (CERF). This was particularly interesting to us because it fit the original challenge idea of dealing with funding gaps in the CBPFs. We hypothesized that UFE funding distribution might be somewhat misaligned with the funding gaps in the CBPFs. If true, this would present an interesting opportunity for the UN to counteract imbalances in donation-based CBPF funding through its more flexible UFE allocations by adjusting the UFE allocation strategy.

We therefore dove deeply into the relevant reports, parsed and cleaned dozens of CSV files, and analyzed the relationship between the UFE measure for crisis severity (the CIRV index), UFE funding allocation, and CBPF funding gaps based on CBPF allocation data. An overview of the datasets and sources we inspected is provided in Appendix Table 3. In the end, we found correlation between the

CIRV index and UFE funding, but both were effectively uncorrelated with the CBPF funding gaps. After a large amount of work, this was a major setback for us.

We decided to stay with the idea of using CERF-related data to inform decisions about other areas of OCHA funding. Next, we tested for a relationship between UFE funding and crisis coverage by the world media, investigating whether there are severe crises that are underrepresented in the media. An example of this news-report analysis is shown in Appendix Figure 1.

We also analyzed UFE allocations across sectors at the level of individual projects to find potential mismatches with CBPF allocations, looking for possible sectoral funding bias in one of the two mechanisms.

Finally, we analyzed UFE funding allocation itself and its relationship to crisis severity, hoping to identify possible mismatches in country and funding allocation that would provide opportunities for improvement in the UFE allocation strategy.

Although we would describe our data analysis skills as quite advanced, these analyses still took a significant amount of time, only for us to realize that they did not produce statistically significant and actionable insights.

While these were not the kind of results that could directly support decision makers, they led us to a very valuable observation. The time spent between obtaining the data and concluding that it would not lead to actionable insight was very long—too long for a decision maker who may need to go through this process regularly.

This directly motivated our final idea: an automated modeling tool that takes the user from their dataset, through feature selection and model setup, to a trained model with interpretable outputs and explicit warnings. A decision maker may still discover that a given dataset does not support actionable conclusions, but the time spent reaching that conclusion can be reduced substantially. This allows fast iteration across datasets and questions. This is what we present in our project. Our highest priority was decision support: enabling quick iteration while also communicating uncertainty, model limitations, and the risks of over-interpreting weak results.

3 System Overview

The platform is built around a simple interactive flow:

1. upload a CSV file,
2. inspect a preview and summary,
3. choose a target and treatment variables,
4. configure per-feature encodings,
5. train a model,
6. review metrics and diagnostics,
7. inspect adaptive pitfall warnings,
8. run single-row inference with uncertainty.

This structure is important for two reasons. First, it reduces friction for non-technical users by turning exploratory modeling into a guided sequence of steps. Second, it keeps the platform flexible across many tabular datasets because the schema is inferred from the uploaded file rather than fixed in advance.

The implementation is split into two main components. The application layer manages upload handling, interactive state, diagnostics rendering, and inference forms, while the modeling layer manages numeric parsing, role inference, encoding, validation, training, and prediction. This separation makes the platform easy to extend with new models or interface elements later.

4 Modeling Pipeline

4.1 Upload and Validation

The system starts from a user-uploaded CSV. It parses the file, creates a dataset representation, and surfaces basic summary information and a preview table. Before training, the platform validates that a target is selected, at least one treatment variable is chosen, the referenced columns exist, and the target is not duplicated among the inputs.

These checks are intentionally lightweight. They do not prove that the modeling setup is meaningful, but they prevent common configuration mistakes that would otherwise mislead users.

4.2 Feature Encoding

A core strength of the platform is that feature processing is not hard-coded. Instead, the user configures encodings per selected treatment column. The current implementation supports the following encoding families:

Encoding	Intended Use
<code>numeric_scaled</code>	Continuous numeric values requiring normalization
<code>numeric_raw</code>	Continuous numeric values kept in original scale
<code>one_hot</code>	Nominal categorical variables
<code>ordinal</code>	Ordered categorical variables
<code>multi_hot_literal</code>	Multi-label features already expressed as literals
<code>multi_hot_delimited</code>	Multi-label features stored as delimiter-separated strings

This design is central to the claim that the platform generalizes widely. Many tabular systems fail not because a model is unavailable, but because the data representation step is too rigid.

4.3 Models and Prediction

The current platform supports two model families: Bayesian Ridge and Random Forest. This is a pragmatic choice. Bayesian Ridge provides a stable linear baseline and a natural notion of predictive uncertainty, while Random Forest adds nonlinear capacity and interaction handling with relatively robust behavior on many tabular problems.

After training, the platform exposes holdout metrics and diagnostics. It also supports single-row inference. For Bayesian Ridge, the interface reports a point estimate and a standard-deviation-based interval. For Random Forest, interval estimates are derived from the distribution of predictions across trees. These intervals are not presented as guarantees, but as uncertainty signals that help users judge how cautious they should be.

5 Pitfalls, Uncertainty, and Honest Output Design

A central part of the project is the *Pitfalls* panel. Many tools show users only a metric or chart and assume they can interpret it correctly. Our platform adds an explanatory layer that reacts to the trained model and the current UI state.

Before training, the panel provides guidance. After training, it surfaces warnings based on fit quality and model complexity. If the user changes the model, target, features, or encodings after training, the panel marks the result as stale and prompts retraining. This helps prevent a common failure mode in interactive tools: users believing an old model still corresponds to their current selections.

This design also addresses a core success criterion of the project: the system should be honest about what it does not know. Rather than presenting outputs with false precision, the platform surfaces intervals, warnings, and stale-state alerts. The goal is not to claim certainty, but to help users understand when a result is only tentative.

6 Decision-Support Use in a Real Workflow

The platform is intended for a humanitarian coordinator or analyst who has data but lacks the time or technical background to build a custom ML pipeline. In a realistic workflow, such a user would upload a dataset, define a target of interest, select plausible explanatory variables, and train a lightweight model to explore whether the data contains a usable signal.

The output is useful not because it gives a final answer, but because it supports the next step in decision-making. A user may discover that the dataset has little predictive structure, which is already informative. Alternatively, they may obtain a tentative estimate, inspect diagnostics, and use the result to prioritize further manual review, request better data, or compare scenarios. In that sense, the platform produces outputs that can guide action: it helps users decide what deserves further attention, where uncertainty is high, and where the data is too weak for confident use.

This is why the system is explicitly framed as *decision support* rather than automated decision-making. It does not replace human judgment, policy considerations, or domain knowledge. It helps a person reason faster and more carefully with available data.

7 Assumptions and Limitations

Our current approach makes several important assumptions:

1. **Tabular assumption.** The uploaded data is meaningfully representable as rows and columns.
2. **Supervised framing.** A useful target column can be identified by the user for prediction.
3. **Row comparability.** Rows correspond to comparable units of analysis.
4. **Static snapshot assumption.** The current workflow assumes a snapshot-style table rather than an explicitly temporal relational schema.
5. **Basic data quality.** Parsing and modeling assume the CSV is not dominated by severe corruption, ambiguity, or contradictory semantics.
6. **User intent assumption.** The user is expected to choose a reasonable target and plausible treatments, even if the platform helps guard against configuration mistakes.

Further, the model family is intentionally small. The platform currently supports Bayesian Ridge and Random Forest, which is a strong starting point but not the full spectrum of tabular ML. The evaluation setup is exploratory rather than deployment-grade: holdout diagnostics are useful, but they do not replace stronger validation, calibration, or robustness analysis.

A further limitation is semantic: the platform can learn predictive relationships, but it does not automatically know whether those relationships are causal, ethically appropriate, or operationally safe to use.

8 Generalization and Future Work

The main strength of the project is not that it solves one narrow task perfectly, but that it provides a reusable platform core for many tabular-data settings. The same design can support humanitarian, administrative, operational, or evaluation datasets as long as they are representable as CSV tables.

The architecture also makes extension straightforward. Future versions could add classification models, stronger tabular learners, richer explanation methods, fairness checks, better missing-data handling, temporal support, geospatial modules, and domain-specific constraints for humanitarian workflows. This is why we see the current system as a strong foundation: it already addresses the core idea and can be expanded substantially without redesigning the product from scratch.

9 Conclusion

We presented *Not a Dashboard*, an interactive platform developed for the UNOCHA project to help users quickly explore arbitrary CSV data with basic machine learning. The platform supports upload-

driven analysis, configurable preprocessing, lightweight model training, diagnostics, uncertainty-aware prediction, and adaptive pitfall messaging. Its most important contribution is not a claim of perfect predictive performance, but a practical decision-support workflow for non-technical users.

The system is intentionally honest about uncertainty and limited knowledge. It is designed to help a user decide what to inspect next, what to question, and when a result should not be trusted too much. In this sense, it satisfies the central goal of the project: enabling useful, cautious, and extensible ML-assisted exploration for real decision-support settings.

A Appendix

Table 1: Core runtime components and responsibilities

Component	Responsibility
dashboard/app.py	Orchestrates upload, selection UI, dynamic encoding configuration, training callbacks, diagnostics rendering, pitfalls refresh, inference form generation, and prediction display.
dashboard/data_model.py	Implements numeric coercion, column-role inference, setup validation, fitted feature specifications, dynamic transformation, model training, and uncertainty-aware prediction.
DynamicModelBundle	Stores the fitted model, target, fitted feature specs, encoded feature names, metrics, diagnostics, metadata, and uncertainty parameter for downstream interaction.

Table 2: Strengths, assumptions, and limitations

Aspect	Summary
Strengths	Generic CSV ingestion, configurable feature encoding, lightweight ML workflow, uncertainty-aware prediction, adaptive warnings, and clear applicability to exploratory decision support.
Assumptions	Tabular data is meaningful, rows are comparable, a sensible target can be defined, and users choose features that are relevant and available in practice.
Limitations	Small model family, exploratory validation only, lightweight explanation layer, no automatic causal reasoning, and uncertainty that should be treated as informative rather than definitive.
Misuse Risks	Users may over-interpret model quality, confuse association with causation, or treat uncertain outputs as decision rules. The platform is therefore framed as decision support, not automation.
Generalization	Strong across many CSV-based tabular settings, with a modular architecture that can be extended with additional models, diagnostics, and domain-specific logic.

Table 3: Overview of explored datasets and data sources.

Dataset / Source	Time Period	Main Variables Used	Notes / Intended Question
Worldwide UFE funding	2017–2026	Country, year, allocation amount, UFE allocation metadata	Tracked global UFE allocation volumes and trends as a baseline for comparison with other OCHA funding mechanisms.
Worldwide CBPF funding	2017–2026	Country, year, allocation amount, pooled-fund totals	Measured global CBPF funding patterns and compared them with UFE funding and inferred funding gaps.
CIRV index calculation	2016–2025	CIRV components, severity indicators, country-year scores	Reconstructed the crisis-severity measure used in UFE analysis and tested how strongly it aligned with observed allocations.
CBPF funding per sector	2021–2025	Sector, country, allocation amount, sectoral shares	Examined whether sector-level CBPF allocations suggested mismatches or bias across humanitarian sectors.
Humanitarian Response Plans for 14 countries via OCHA FTS	2021–2026	HRP requirements, funded amounts, coverage ratios, country-year values	Derived funding-gap and coverage measures from OCHA FTS data for cross-country comparison.
GDELT project for world-wide media coverage	Project-wide	News volume, country mentions, lagged media-coverage signals	Studied whether severe crises were underrepresented in global media coverage relative to funding and severity.
Global CBPF funded projects	2015–2025	Project descriptions, countries, funding amounts, matched sector labels	Required a major processing step: project descriptions were matched to OCHA target sectors using the CLAUDE API.

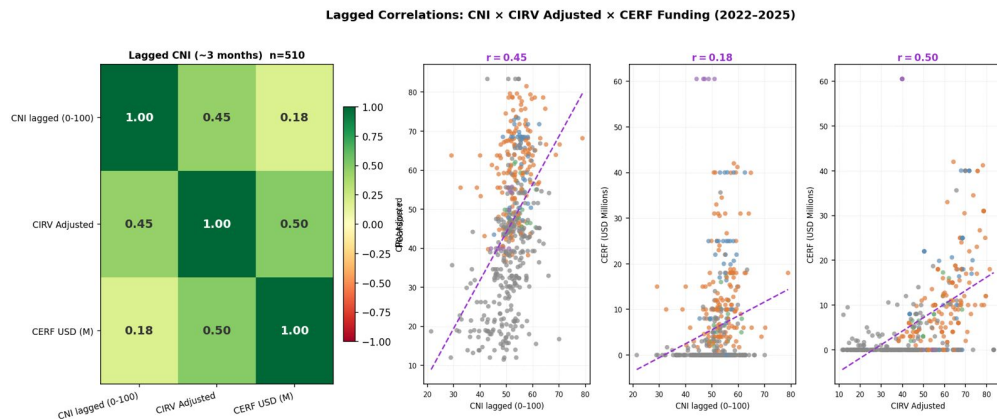


Figure 1: Example result from our news-report analysis. The figure shows lagged correlations between lagged CNI, adjusted CIRV, and CERF funding over 2022–2025. While some pairwise relationships are visible, this line of analysis also illustrates the broader challenge we encountered: substantial analytical effort did not reliably translate into statistically strong, decision-ready insight.